PostgreSQL
COMPLETE NOTES

Detailed Premium Edition • 30–40 Pages
Database Fundamentals → Advanced SQL → Optimization

Parts 1 – 5 | From Basics to Interview-Ready

Expanded Explanations • More Examples • Diagrams
Interview Tips • Practice Sets • Real-World Queries

Data Types • Constraints • CRUD • Filtering • Aggregation
JOINs • Subqueries • Window Functions • CTEs • Indexes • Transactions
Functions • Triggers • JSONB • Arrays • Query Optimization • Design

Table of Contents

Part 1 — Fundamentals, Data Types, Constraints & Basic SQL
Part 2 — Data Retrieval, Filtering, Sorting & Aggregate Functions
Part 3 — JOINs, Relationships & Subqueries
Part 4 — Window Functions, CTEs, Views, Indexes & Transactions
Part 5 — Functions, Procedures, Triggers, JSONB, Arrays & Optimization

1 PART 1 — Fundamentals, Data Types, Constraints & Basic SQL

1.1 PostgreSQL Kya Hai?

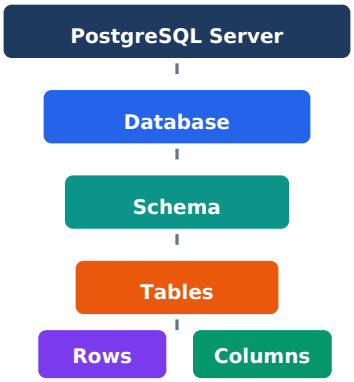
PostgreSQL ek powerful, open-source object-relational database management system (ORDBMS) hai. Ye ACID-compliant hai, complex queries, foreign keys, triggers, views, transactional integrity aur advanced data types support karta hai. Industry mein isko reliability aur standards compliance ke liye prefer kiya jata hai.

SQL (Structured Query Language) database se communicate karne ki standard language hai. PostgreSQL SQL standards ko closely follow karta hai aur bahut saare extensions bhi deta hai.

Core Terminology

Term	Meaning	Example / Note
PostgreSQL	Open-source RDBMS	Often called Postgres
SQL	Language to talk to DB	SELECT, INSERT, UPDATE…
Database	Related data ka collection	ecommerce_db
Schema	Objects ka namespace	public, sales, hr
Table	Rows + columns structure	customers, orders
Row (Tuple)	Ek complete record	One customer
Column (Attribute)	Ek field / property	email, price
Primary Key	Row uniquely identify	customer_id
Foreign Key	Do tables ka link	orders.customer_id
Index	Fast lookup structure	idx_email
Constraint	Data integrity rule	NOT NULL, UNIQUE, CHECK

Database Hierarchy



Server → Database → Schema → Tables → Rows + Columns

Ek PostgreSQL server par multiple databases ho sakte hain. Har database ke andar multiple schemas. Default schema public hota hai. Tables schema ke andar rehte hain.

1.2 Numeric Data Types — Deep Dive

Column ka data type decide karta hai ki usme kaunsi type ki value store hogi, kitna storage lagega, aur kaunse operations allowed hain.

Data Type	Storage	Range / Precision	Typical Use
SMALLINT	2 bytes	−32,768 to 32,767	Age, small counts
INTEGER / INT	4 bytes	−2.1B to 2.1B	IDs, quantities
BIGINT	8 bytes	Very large range	Population, big counters
NUMERIC(p,s)	Variable	Exact, user-defined	Money, precise calc
DECIMAL(p,s)	Variable	Same as NUMERIC	Same as NUMERIC
REAL	4 bytes	~6 decimal digits	Scientific approx

Data Type	Storage	Range / Precision	Typical Use
DOUBLE PRECISION	8 bytes	~15 decimal digits	High-precision float
SERIAL	4 bytes	Auto-increment INT	Legacy auto ID
BIGSERIAL	8 bytes	Auto-increment BIGINT	Large auto IDs

NUMERIC(p, s) explained

```sql
NUMERIC(10, 2)
-- p = 10  → total maximum significant digits
-- s = 2   → digits after decimal point
-- Valid: 12345678.90   (10 digits total)
-- Invalid: 123456789.01  (too many digits)
```

TIP

Financial / money values ke liye hamesha NUMERIC ya DECIMAL use karo. REAL aur DOUBLE floating-point approximation use karte hain — paise ke calculation mein rounding errors aa sakte hain.

Use-case mapping

```sql
age            → INTEGER
quantity       → INTEGER
population     → BIGINT
product_price  → NUMERIC(10,2)
salary         → NUMERIC(12,2)
latitude       → DOUBLE PRECISION
```

1.3 Character / String Data Types

Type	Length	Padding	Best For
CHAR(n)	Fixed n	Yes (spaces)	Fixed codes (country_code CHAR(2))
VARCHAR(n)	Max n	No	Names, emails, titles
TEXT	Unlimited*	No	Descriptions, comments, long text

*Practical limit system resources par depend karta hai. PostgreSQL mein VARCHAR aur TEXT performance almost same hoti hai — length limit mainly validation ke liye useful hai.

```sql
name         VARCHAR(100)
email        VARCHAR(150)
description  TEXT
country_code CHAR(2)
phone        VARCHAR(20)
```

NOTE

Modern apps mein aksar VARCHAR(n) ya TEXT dono acceptable hain. Agar business rule se maximum length fixed hai (jaise email 255), to VARCHAR use karo for clarity.

1.4 Boolean

BOOLEAN sirf teen values accept karta hai: TRUE, FALSE, NULL (unknown).

```sql
is_active BOOLEAN DEFAULT TRUE
-- Accepted: TRUE, FALSE, 't', 'f', 'yes', 'no', '1', '0' (with care)
-- Prefer explicit TRUE / FALSE in application code
```

1.5 Date & Time Types

Type	Stores	Example	Notes
DATE	Date only	2026-09-21	No time component
TIME	Time only	14:30:00	Optional precision

Type	Stores	Example	Notes
TIMESTAMP	Date + time	2026-09-21 14:30:00	No timezone
TIMESTAMPTZ	Instant in time	2026-09-21 14:30:00+05:30	Timezone aware
INTERVAL	Duration	2 days 3 hours	Date arithmetic

```sql
birth_date   DATE
login_time   TIME
created_at   TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
updated_at   TIMESTAMPTZ
duration     INTERVAL
```

TIP

Jahan actual moment in time matter karta hai (logs, events, orders), TIMESTAMPTZ prefer karo. TIMESTAMP without TZ timezone conversion issues de sakta hai multi-region apps mein.

1.6 UUID

UUID = Universally Unique Identifier. Sequential integer IDs ki jagah globally unique, hard-to-guess identifiers.

```sql
-- Example value
550e8400-e29b-41d4-a716-446655440000

user_id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Common use: distributed systems, public API resource IDs, merge-friendly primary keys across databases. Extension pgcrypto ya built-in gen_random_uuid() (PG 13+) se generate karte hain.

1.7 ENUM

ENUM sirf predefined values allow karta hai — data integrity ke liye useful.

```sql
CREATE TYPE order_status AS ENUM (
    'pending',
    'shipped',
    'delivered',
    'cancelled'
);

-- Column
status order_status DEFAULT 'pending'

-- Valid inserts: 'pending', 'shipped', ...
-- Invalid: 'processing'  (unless added to type)
```

⚠ IMPORTANT

ENUM values later add karna possible hai (ALTER TYPE ... ADD VALUE), lekin remove/reorder karna mushkil hota hai. Kabhi-kabhi lookup table + FK zyada flexible hota hai.

1.8 JSON / JSONB

Type	Storage	Query / Index	When to use
JSON	Text, preserves format	Slower	Audit log of exact input
JSONB	Binary, decomposed	Fast + GIN index	Most application use

```sql
metadata JSONB
-- Example data:
{"color": "black", "size": "XL", "brand": "Nike"}
```

TIP

General rule: JSONB prefer karo jab tumhe query, filter ya index JSON fields par chahiye.

1.9 Arrays

```SQL
skills TEXT[]
scores INTEGER[]

-- Insert
ARRAY['Python', 'SQL', 'Machine Learning']
-- or '{Python,SQL,Machine Learning}'

-- Access (1-based index)
skills[1]   -- first element
-- Search
WHERE 'Python' = ANY(skills)
-- Expand to rows
SELECT UNNEST(skills) FROM students;
```

1.10 Other Useful Types (Quick)

Type	Use
BYTEA	Binary data (files, images as bytes)
MONEY	Currency display — prefer NUMERIC for calculations
INET / CIDR	IP addresses / networks
MACADDR	MAC addresses
XML	XML documents
POINT / geometric	Simple geometry

Most important types to master first

INTEGER • BIGINT • NUMERIC • VARCHAR • TEXT • BOOLEAN • DATE • TIMESTAMP/TIMESTAMPTZ • UUID • JSONB • ARRAY

1.11 Constraints — Complete Guide

Constraints database level par rules enforce karte hain taaki invalid data enter na ho. Application logic ke saath yeh second line of defense hain.

Constraint	Purpose	Allows NULL?
NOT NULL	Value compulsory	No
UNIQUE	No duplicate values	Yes (multiple NULLs OK)
PRIMARY KEY	Unique row identity	No
FOREIGN KEY	Referential integrity	Depends
CHECK	Custom condition	Yes (unless blocked)
DEFAULT	Auto value if omitted	—

NOT NULL

```sql
name VARCHAR(100) NOT NULL
-- Allowed: 'Gaurav'
-- Rejected: NULL
```

Use when field business-critical hai (name, email, amount, etc.).

UNIQUE

```sql
email VARCHAR(150) UNIQUE
-- a@gmail.com  ✓
-- a@gmail.com again  ✗
-- NULL, NULL  ✓ (NULLs are not considered equal)
```

i NOTE

PostgreSQL UNIQUE constraint multiple NULL values allow karta hai because NULL ≠ NULL in uniqueness comparison.

PRIMARY KEY

Table ki har row ko uniquely identify karta hai. Internally UNIQUE + NOT NULL. Normally ek table mein ek hi primary key constraint hota hai (single column ya composite).

```sql
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    name VARCHAR(100)
);

-- Composite primary key example
CREATE TABLE order_items (
    order_id   INTEGER,
    product_id INTEGER,
    quantity   INTEGER,
    PRIMARY KEY (order_id, product_id)
);
```

FOREIGN KEY

Child table ke column ko parent table ke primary (ya unique) key se link karta hai. Referential integrity maintain hoti hai.

```sql
CREATE TABLE customers (
    customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE orders (
    order_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    amount NUMERIC(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    -- optional: ON DELETE CASCADE / SET NULL / RESTRICT
);
```

Agar customer exist nahi karta to us customer_id ke saath order insert nahi ho sakta (default RESTRICT behavior).

CHECK

```sql
age INTEGER CHECK (age >= 18)
price NUMERIC(10,2) CHECK (price >= 0)
stock INTEGER CHECK (stock >= 0)

CREATE TABLE products (
    product_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) CHECK (price >= 0),
    discount_pct NUMERIC(5,2) CHECK (discount_pct BETWEEN 0 AND 100)
);
```

DEFAULT

```sql
city VARCHAR(50) DEFAULT 'Delhi'
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP

INSERT INTO customers (name) VALUES ('Gaurav');
-- city automatically = 'Delhi' if column has DEFAULT
```

Modern Auto-Generated IDs

```sql
-- Preferred modern way (PostgreSQL 10+)
id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY

-- GENERATED BY DEFAULT AS IDENTITY  → allows manual override
-- SERIAL / BIGSERIAL                → older convenience (still works)
```

INTERVIEW TIP

Interview mein pucho to: SERIAL ke peeche sequence + DEFAULT nextval() hota hai. GENERATED ALWAYS AS IDENTITY SQL-standard aur clearer intent dikhata hai.

1.12 Creating Databases & Tables

```sql
CREATE DATABASE ecommerce_db;
-- DROP DATABASE ecommerce_db;   -- permanent, careful!

CREATE TABLE products (
    product_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category     VARCHAR(50) NOT NULL,
    price        NUMERIC(10,2) CHECK (price >= 0),
    stock        INTEGER CHECK (stock >= 0),
    is_active    BOOLEAN DEFAULT TRUE,
    created_at   TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

1.13 INSERT — Detailed

```sql
-- Single row
INSERT INTO products (product_name, category, price, stock)
VALUES ('Laptop', 'Electronics', 65000, 10);

-- Multiple rows
INSERT INTO products (product_name, category, price, stock)
VALUES
    ('Laptop', 'Electronics', 65000, 10),
    ('Mouse',  'Electronics', 800, 50),
    ('Chair',  'Furniture', 5000, 20);

-- Insert with RETURNING (get generated ID back)
INSERT INTO products (product_name, category, price, stock)
VALUES ('Keyboard', 'Electronics', 1500, 30)
RETURNING product_id, product_name;
```

1.14 SELECT Basics

```sql
SELECT * FROM products;                    -- all columns
SELECT product_name, price FROM products;  -- specific
SELECT product_name AS name, price AS mrp FROM products;  -- alias
```

1.15 WHERE — Filtering Rows

Comparison operators: = <> != > < >= <=

```sql
SELECT * FROM products WHERE price > 5000;
SELECT * FROM products WHERE category = 'Electronics' AND price > 1000;
SELECT * FROM products WHERE category = 'Electronics' OR category = 'Furniture';
SELECT * FROM products WHERE category IN ('Electronics', 'Furniture');
SELECT * FROM products WHERE price BETWEEN 1000 AND 10000;
SELECT * FROM products WHERE product_name LIKE 'L%';      -- starts with L
SELECT * FROM products WHERE product_name ILIKE 'laptop'; -- case-insensitive
SELECT * FROM products WHERE category IS NULL;
SELECT * FROM products WHERE category IS NOT NULL;
```

AVOID

WHERE category = NULL kabhi mat likho. NULL comparison ke liye hamesha IS NULL / IS NOT NULL.

1.16 UPDATE & DELETE

```sql
UPDATE products
SET price = 60000, stock = 15
WHERE product_id = 1;

DELETE FROM products WHERE product_id = 3;
-- DELETE FROM products;   -- ALL rows (dangerous without WHERE)
```

⚠ IMPORTANT

UPDATE / DELETE se pehle WHERE condition double-check karo. Production mein pehle SELECT same WHERE se dekho ki kaunsi rows affect hongi.

DELETE vs TRUNCATE vs DROP

Command	Rows removed?	Table structure?	Rollback?	Speed
DELETE	Selected or all	Kept	Yes (in txn)	Slower (logs)
TRUNCATE	All rows	Kept	Yes (mostly)	Very fast
DROP TABLE	All + structure	Removed	Yes (in txn)	Fast

1.17 ALTER TABLE

```sql
ALTER TABLE products ADD COLUMN brand VARCHAR(100);
ALTER TABLE products RENAME COLUMN brand TO manufacturer;
ALTER TABLE products ALTER COLUMN product_name TYPE VARCHAR(200);
ALTER TABLE products DROP COLUMN manufacturer;
ALTER TABLE products RENAME TO product_details;
ALTER TABLE products ADD CONSTRAINT chk_stock CHECK (stock >= 0);
ALTER TABLE products ALTER COLUMN is_active SET DEFAULT TRUE;
```

1.18 SQL Command Categories

Category	Full Form	Commands	Purpose
DDL	Data Definition	CREATE, ALTER, DROP, TRUNCATE	Structure
DML	Data Manipulation	INSERT, UPDATE, DELETE	Change data
DQL	Data Query	SELECT	Read data
DCL	Data Control	GRANT, REVOKE	Permissions
TCL	Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	Transactions

1.19 Part 1 Quick Revision Sheet

```sql
Hierarchy: Server → DB → Schema → Table → Rows/Columns
Types: INT, BIGINT, NUMERIC, VARCHAR, TEXT, BOOL, DATE, TIMESTAMPTZ, UUID, JSONB, ARRAY
Constraints: PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT
CRUD: CREATE / INSERT / SELECT / UPDATE / DELETE / ALTER / DROP / TRUNCATE
Auto ID: GENERATED ALWAYS AS IDENTITY
NULL checks: IS NULL / IS NOT NULL (never = NULL)
```

PRACTICE

Practice: customers table banao — customer_id (identity PK), name NOT NULL, email UNIQUE, phone, city DEFAULT 'Delhi', age CHECK >= 18, is_active DEFAULT TRUE, created_at TIMESTAMPTZ default now. 8–10 rows insert karo. Phir SELECT, WHERE, UPDATE, DELETE, ALTER practice karo.

2 PART 2 — Retrieval, Filtering, Sorting & Aggregates

2.1 SELECT Mastery

```sql
SELECT * FROM customers;
SELECT name, email, city FROM customers;
SELECT name AS customer_name, email AS customer_email FROM customers;
-- AS optional:
SELECT name customer_name FROM customers;
```

2.2 DISTINCT

Duplicate values hataata hai. Multiple columns par DISTINCT unke combination ko unique maanta hai.

```sql
SELECT DISTINCT city FROM customers;
SELECT DISTINCT city, age FROM customers;  -- unique (city, age) pairs
```

2.3 WHERE — Comparison & Logical Operators

Operator	Meaning	Example
=	Equal	city = 'Delhi'
<> or !=	Not equal	status <> 'cancelled'
> < >= <=	Comparisons	price >= 5000
AND	All true	dept = 'IT' AND salary > 50000
OR	At least one true	dept = 'IT' OR dept = 'HR'
NOT	Negate	NOT dept = 'IT'

```sql
SELECT * FROM employees
WHERE department = 'IT'
  AND (salary > 50000 OR city = 'Delhi');
-- Parentheses important for mixed AND/OR
```

2.4 IN, BETWEEN, LIKE, ILIKE

```sql
-- IN
WHERE city IN ('Delhi', 'Agra', 'Mumbai');
WHERE city NOT IN ('Delhi', 'Agra');

-- BETWEEN (inclusive)
WHERE price BETWEEN 1000 AND 5000;
-- same as: price >= 1000 AND price <= 5000

-- LIKE patterns
WHERE name LIKE 'A%'     -- starts with A
WHERE name LIKE '%a'     -- ends with a
WHERE name LIKE '%an%'   -- contains an
WHERE name LIKE '_man'   -- exactly one char then man
-- % = zero or more chars    _ = exactly one char

-- ILIKE = case-insensitive LIKE (PostgreSQL specific)
WHERE name ILIKE 'gaurav';  -- matches Gaurav, GAURAV, gaurav...
```

2.5 NULL Handling

NULL = missing / unknown value. 0, empty string '', aur NULL teen alag cheezein hain.

```sql
WHERE phone IS NULL;
WHERE phone IS NOT NULL;
-- WHERE phone = NULL;   -- WRONG, always unknown
```

⚠ AVOID

NULL ke saath arithmetic ya comparison ka result usually NULL (unknown) hota hai. Aggregates mein COUNT(col) NULLs skip karta hai; COUNT(*) nahi.

2.6 ORDER BY, LIMIT, OFFSET

```sql
SELECT * FROM products ORDER BY price ASC;   -- low to high
SELECT * FROM products ORDER BY price DESC;  -- high to low
SELECT * FROM employees
ORDER BY department ASC, salary DESC;        -- multi-column

-- Top N
SELECT * FROM products ORDER BY price DESC LIMIT 5;

-- Pagination
SELECT * FROM products LIMIT 10 OFFSET 0;   -- page 1
SELECT * FROM products LIMIT 10 OFFSET 10;  -- page 2
SELECT * FROM products LIMIT 10 OFFSET 20;  -- page 3
```

ℹ NOTE

ORDER BY column position bhi chal sakta hai (ORDER BY 2 DESC), lekin readability ke liye column name better hai.

2.7 Aggregate Functions — Deep

Function	Purpose	NULL behavior
COUNT(*)	Total rows	Counts all rows
COUNT(col)	Non-null values	Skips NULL
COUNT(DISTINCT col)	Unique non-null	Skips NULL + dups
SUM(col)	Total	Skips NULL
AVG(col)	Average	Skips NULL
MIN(col) / MAX(col)	Min / Max	Skips NULL

```sql
SELECT COUNT(*) FROM customers;
SELECT COUNT(phone) FROM customers;          -- ignores NULL phones
SELECT COUNT(DISTINCT city) FROM customers;
SELECT SUM(price), AVG(price), MIN(price), MAX(price) FROM products;
SELECT AVG(salary) FROM employees WHERE department = 'IT';
```

2.8 GROUP BY

Same values ko groups mein baantta hai, phir har group par aggregate lagta hai.

```sql
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;

SELECT department, SUM(salary) AS total_salary, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;

-- Multiple columns
SELECT department, city, COUNT(*)
FROM employees
GROUP BY department, city;
```

⚠ **IMPORTANT**

GROUP BY ke saath SELECT mein sirf grouped columns ya aggregate functions hone chahiye. Random non-aggregated column error dega.

2.9 HAVING

Groups ko filter karta hai (WHERE individual rows ko).

SQL

```
SELECT department, COUNT(*) AS cnt
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;

SELECT department, AVG(salary) AS avg_sal
FROM employees
WHERE salary > 30000          -- filter rows first
GROUP BY department
HAVING AVG(salary) > 50000;   -- then filter groups
```

WHERE	HAVING
Rows filter (before grouping)	Groups filter (after grouping)
Cannot use aggregate directly	Designed for aggregate conditions
Cheaper — less data to group	Runs after aggregation

2.10 CASE Expression

```SQL
SELECT name, salary,
    CASE
        WHEN salary >= 80000 THEN 'High'
        WHEN salary >= 50000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_level
FROM employees;

-- Simple CASE
SELECT product_name,
    CASE category
        WHEN 'Electronics' THEN 'Tech'
        WHEN 'Furniture' THEN 'Home'
        ELSE 'Other'
    END AS category_group
FROM products;
```

2.11 COALESCE & NULLIF

```SQL
-- First non-NULL value
SELECT name, COALESCE(phone, email, 'No Contact') AS contact
FROM customers;

-- NULLIF: return NULL if both args equal
SELECT NULLIF(10, 10);   -- NULL
SELECT NULLIF(10, 5);    -- 10

-- Classic: avoid division by zero
SELECT sales / NULLIF(quantity, 0) AS unit_price FROM products;
```

2.12 String Functions

```SQL
UPPER(name)          -- GAURAV
LOWER(name)          -- gaurav
LENGTH(name)         -- character count
TRIM(name)           -- remove leading/trailing spaces
CONCAT(a, ' ', b)    -- or a || ' ' || b
SUBSTRING(name,1,3)  -- first 3 chars
REPLACE(name,'a','x')
LEFT(name, 3) / RIGHT(name, 3)
POSITION('a' IN name)
```

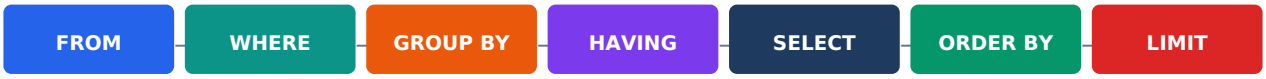
2.13 Date / Time Functions

```SQL
SELECT CURRENT_DATE;
SELECT CURRENT_TIMESTAMP;
SELECT NOW();

SELECT EXTRACT(YEAR FROM created_at);
SELECT EXTRACT(MONTH FROM created_at);
SELECT EXTRACT(DAY FROM created_at);
SELECT EXTRACT(HOUR FROM created_at);

SELECT CURRENT_DATE + INTERVAL '7 days';
SELECT CURRENT_DATE - INTERVAL '30 days';
SELECT AGE(CURRENT_DATE, birth_date);   -- interval age
SELECT DATE_TRUNC('month', order_date); -- first day of month
```

2.14 Logical Query Execution Order (Must Memorize)



FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Is order ko samajhna complex queries aur interview questions ke liye critical hai. Alias SELECT phase mein bante hain, isliye WHERE mein alias use nahi kar sakte (HAVING / ORDER BY mein often kar sakte ho).

```sql
SELECT department, AVG(salary) AS average_salary
FROM employees
WHERE salary > 30000
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY average_salary DESC
LIMIT 3;
```

2.15 Complete Example Walkthrough

employees: Rahul IT 60k Delhi, Priya HR 50k Mumbai, Aman IT 80k Agra, Neha Finance 70k Delhi, Ravi IT 45k Delhi, Simran HR 60k Agra.

```sql
-- Dept-wise avg salary, only depts with avg > 55k, highest first
SELECT department, AVG(salary) AS average_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY average_salary DESC;
```

2.16 Part 2 Quick Revision

```sql
SELECT → DISTINCT → WHERE (AND/OR/NOT/IN/BETWEEN/LIKE/ILIKE/IS NULL)
ORDER BY → LIMIT / OFFSET
COUNT SUM AVG MIN MAX → GROUP BY → HAVING
CASE → COALESCE → NULLIF
String & Date functions
Execution order: FROM WHERE GROUP BY HAVING SELECT ORDER BY LIMIT
```

PRACTICE

Practice (employees table): 1) Highest salary employee 2) Lowest 3) Top 5 paid 4) Unique departments 5) IT + HR 6) Salary 40k–70k 7) Count per dept 8) Total salary per dept 9) Avg per dept 10) Depts with >3 employees 11) Uppercase names 12) COALESCE phone 13) CASE High/Medium/Low 14) Salary DESC 15) Second page of 10 rows.

3 PART 3 — JOINs, Relationships & Subqueries

3.1 Why JOINs Exist

Real databases normalized hote hain — data multiple tables mein related form mein store hota hai. Ek query mein related data chahiye to tables ko JOIN karna padta hai.

Example: customers (id, name, city) + orders (id, customer_id, product, amount). Customer name + order details chahiye → JOIN on customer_id.

Relationship Types

Type	Example	How modeled
One-to-One	Person ↔ Passport	PK-FK, often unique FK
One-to-Many	Customer → Orders	FK on many side
Many-to-Many	Students ↔ Courses	Junction / bridge table

3.2 Foreign Key Relationship Recap

```sql
CREATE TABLE customers (
    customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
CREATE TABLE orders (
    order_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    amount NUMERIC(10,2)
);
```

3.3 JOIN Types — Visual + Examples



INNER JOIN

Sirf matching rows. Non-matching dono taraf se drop.

```sql
SELECT c.name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
-- JOIN alone = INNER JOIN
```

LEFT JOIN (LEFT OUTER JOIN)

Left table ki saari rows + matching right. No match → right columns NULL.

```sql
SELECT c.name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
-- Customers without orders still appear (order_id NULL)
```

RIGHT JOIN & FULL OUTER JOIN

RIGHT = all right + matching left. FULL = all from both (NULLs where no match). Practice mein LEFT JOIN zyada common hai; tables ka order badal kar RIGHT ko LEFT bana sakte ho.

CROSS JOIN

Cartesian product — har row × har row. 3 × 4 = 12 rows. Useful for combinations (sizes × colors), lekin accidental use se result explode ho sakta hai.

```sql
SELECT * FROM products CROSS JOIN colors;
```

SELF JOIN

Same table ko alias se do baar join — hierarchical data (employee-manager).

```sql
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

3.4 Multiple JOINs

```sql
SELECT c.name AS customer, o.order_id, p.product_name, oi.quantity
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p     ON oi.product_id = p.product_id;
```

3.5 JOIN + WHERE / GROUP BY

```sql
-- Filter after join
SELECT c.name, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.amount > 50000;

-- Aggregate after join
SELECT c.customer_id, c.name, SUM(o.amount) AS total_spending
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spending DESC;
```

3.6 Classic Pattern: Customers With No Orders

```sql
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

INTERVIEW TIP

Ye pattern bahut common interview question hai. LEFT JOIN + WHERE right-key IS NULL = anti-join (rows in A not in B).

COUNT with LEFT JOIN

```sql
SELECT c.name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
-- Use COUNT(o.order_id) not COUNT(*) so zero-order customers show 0
```

ON vs WHERE with LEFT JOIN

⚠ **IMPORTANT**

Filter condition ON clause mein → left rows preserve. Same condition WHERE mein → unmatched left rows filter out ho sakti hain (INNER jaisa behave). Interviewers is difference ko test karte hain.

3.7 Subqueries

Ek query ke andar doosri query. Inner query pehle evaluate hoti hai (usually).

Scalar Subquery (returns single value)

```sql
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

SELECT * FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

IN / NOT IN

```sql
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
-- WARNING: NOT IN + NULLs in subquery can return unexpected empty result
```

EXISTS / NOT EXISTS (often preferred)

```sql
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

SELECT * FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

TIP

EXISTS sirf existence check karta hai (true/false). Correlated subquery hai — outer row ke hisaab se inner chalega. NOT EXISTS usually NOT IN se safer hota hai jab NULLs possible hon.

3.8 CTE — Common Table Expression

WITH clause se temporary named result set. Query readable aur modular banta hai.

```sql
WITH high_value_orders AS (
    SELECT * FROM orders WHERE amount > 50000
)
SELECT * FROM high_value_orders;

WITH customer_sales AS (
    SELECT customer_id, SUM(amount) AS total_spending
    FROM orders GROUP BY customer_id
)
SELECT * FROM customer_sales WHERE total_spending > 100000;

-- Multiple CTEs
WITH sales AS (...),
     top_customers AS (SELECT * FROM sales WHERE total_spending > 100000)
SELECT * FROM top_customers;
```

Recursive CTE (basic idea)

Hierarchical data (org chart, category tree, folder structure) ke liye. Base case + recursive member UNION ALL se.

```sql
WITH RECURSIVE employee_tree AS (
    -- base: top-level (no manager)
    SELECT employee_id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    -- recursive: children of previous level
    SELECT e.employee_id, e.name, e.manager_id, et.level + 1
    FROM employees e
    JOIN employee_tree et ON e.manager_id = et.employee_id
)
SELECT * FROM employee_tree;
```

3.9 JOIN vs Subquery

JOIN	Subquery
Natural for combining related tables	Natural for filtering / existence
Clear relational combination	Can be more concise for some filters
Often used in reporting	EXISTS / scalar comparisons

Performance ka fixed rule nahi — planner, indexes, data size decide karte hain. EXPLAIN ANALYZE se verify karo.

3.10 Interview Query Patterns

```sql
-- 1. Customers who ordered
SELECT DISTINCT c.* FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;

-- 2. Customers who never ordered
SELECT c.* FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- 3. Total spending per customer
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;

-- 4. Above average salary
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- 5. Has at least one order (EXISTS)
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

3.11 Part 3 Quick Revision

```sql
INNER / LEFT / RIGHT / FULL / CROSS / SELF JOIN
ON = join condition    WHERE = filter (careful with LEFT)
Anti-join: LEFT JOIN ... WHERE right.pk IS NULL
Subquery: scalar, IN, EXISTS / NOT EXISTS
CTE: WITH name AS (SELECT ...) SELECT ... FROM name
Recursive CTE for hierarchies
```

PRACTICE

Practice: 1) Customer + orders 2) All customers incl. no orders 3) Never ordered 4) Total spending 5) Highest spenders 6) Order count per customer 7) Orders > 50k 8) Above avg amount 9) EXISTS 10) 3-table JOIN (customer-order-product).

4 PART 4 — Window Functions, CTEs, Views, Indexes & Transactions

4.1 Window Functions — Concept

Window function related rows ke set par calculation karta hai bina rows ko collapse kiye. GROUP BY result ko ek row per group bana deta hai; window har original row ke saath answer deta hai.

```sql
-- Aggregate: one row per department
SELECT department, AVG(salary) FROM employees GROUP BY department;

-- Window: every employee row + dept average
SELECT employee_name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

Syntax: `function() OVER ( [PARTITION BY ...] [ORDER BY ...] [frame] )`

4.2 ROW_NUMBER, RANK, DENSE_RANK

ROW_NUMBER		RANK		DENSE_RANK	
100k	1	100k	1	100k	1
90k	2	90k	2	90k	2
90k	3	90k	2	90k	2
80k	4	80k	4	80k	3

Same salary 90k: ROW_NUMBER gives 2 & 3, RANK gives 2 & 2 then skips to 4, DENSE_RANK gives 2 & 2 then 3

```sql
SELECT employee_name, salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
    RANK()       OVER (ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;

-- Department-wise
ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC)
```

4.3 Top-N per Group (Very Common Pattern)

```sql
WITH ranked AS (
    SELECT employee_name, department, salary,
           ROW_NUMBER() OVER (
               PARTITION BY department ORDER BY salary DESC
           ) AS rn
    FROM employees
)
SELECT * FROM ranked WHERE rn <= 2;
```

INTERVIEW TIP

Window result ko directly WHERE mein use nahi kar sakte (logical order). Isliye CTE ya subquery se wrap karke filter karte hain. Ye top-N-per-group pattern interviews mein bahut aata hai.

4.4 LAG & LEAD

Previous / next row ki value access — time series, running comparisons.

```sql
SELECT sale_date, sales,
    LAG(sales)  OVER (ORDER BY sale_date) AS prev_sales,
    LEAD(sales) OVER (ORDER BY sale_date) AS next_sales,
    sales - LAG(sales) OVER (ORDER BY sale_date) AS diff
FROM daily_sales;

-- Per customer previous order
LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date)
```

4.5 Running Total & Moving Average

```sql
SELECT sale_date, sales,
    SUM(sales) OVER (ORDER BY sale_date) AS running_total,
    AVG(sales) OVER (
        ORDER BY sale_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_3day
FROM daily_sales;
```

Frame clause (ROWS BETWEEN ...) control karta hai window ka exact range. Default often RANGE UNBOUNDED PRECEDING to CURRENT ROW for aggregates with ORDER BY.

4.6 Aggregate vs Window — Summary

Aspect	GROUP BY Aggregate	Window Function
Rows in output	One per group	Same as input
Syntax	GROUP BY	OVER (...)
Use case	Summary reports	Row + context calculation

4.7 Views

Saved query jo table ki tarah use hoti hai. Complex joins / filters ko reuse karne ke liye.

```sql
CREATE VIEW customer_orders AS
SELECT c.customer_id, c.name, o.order_id, o.order_date, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;

SELECT * FROM customer_orders WHERE amount > 10000;

CREATE OR REPLACE VIEW customer_orders AS ...;
DROP VIEW customer_orders;
```

Materialized View

Result physically store hota hai — expensive aggregations ke liye. Refresh explicit chahiye.

```sql
CREATE MATERIALIZED VIEW monthly_sales AS
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS total_sales
FROM orders
GROUP BY 1;

REFRESH MATERIALIZED VIEW monthly_sales;
```

4.8 Indexes — Deep Dive

Index = data structure (usually B-tree) jo specific columns par fast lookup, join, sort enable karta hai. Book ke index jaisa — poori table scan se bachata hai.

```sql
CREATE INDEX idx_customers_email ON customers(email);
CREATE INDEX idx_orders_customer_id ON orders(customer_id);

-- Composite (column order matters!)
CREATE INDEX idx_orders_cust_date ON orders(customer_id, order_date);

-- Unique index
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Partial index (only subset of rows)
CREATE INDEX idx_active_users ON users(email) WHERE is_active = TRUE;

-- Expression index
CREATE INDEX idx_lower_email ON users(LOWER(email));
```

Advantage	Disadvantage
Faster SELECT / JOIN / ORDER BY	Extra disk space
Faster suitable lookups	Slower INSERT / UPDATE / DELETE
Can enforce uniqueness	Too many indexes hurt write performance

⚠ IMPORTANT

Har column par index mat laga do. WHERE, JOIN, ORDER BY mein frequently use hone wale columns prioritise karo. Composite index ka leftmost prefix matter karta hai.

4.9 Transactions & ACID



Atomicity — saari operations succeed ya koi nahi. Consistency — constraints / rules maintain. Isolation — concurrent transactions interfere na karein. Durability — COMMIT ke baad data permanent.

```sql
BEGIN;
UPDATE accounts SET balance = balance - 10000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 10000 WHERE account_id = 2;
COMMIT;     -- both applied
-- ROLLBACK; -- both undone

BEGIN;
UPDATE ...;
SAVEPOINT step1;
UPDATE ...;          -- problem
ROLLBACK TO SAVEPOINT step1;  -- only second undone
COMMIT;
```

Real-world: order place + stock decrement ek transaction mein hona chahiye. Agar stock update fail → order bhi rollback.

4.10 Classic Window / Ranking Interview Queries

```sql
-- Second highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- or
SELECT * FROM (
    SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 2;

-- Highest salary per department
SELECT * FROM (
    SELECT *, RANK() OVER (
        PARTITION BY department ORDER BY salary DESC
    ) AS rnk FROM employees
) t WHERE rnk = 1;

-- Previous order amount per customer
SELECT customer_id, order_date, amount,
    LAG(amount) OVER (
        PARTITION BY customer_id ORDER BY order_date
    ) AS prev_amount
FROM orders;
```

4.11 Part 4 Quick Revision

```sql
Window: OVER (PARTITION BY ... ORDER BY ...)
ROW_NUMBER / RANK / DENSE_RANK / LAG / LEAD / SUM() OVER
Top-N: window + CTE + filter
View = saved query; Materialized View = stored result
Index: B-tree, composite, partial, expression
Transaction: BEGIN COMMIT ROLLBACK SAVEPOINT
ACID: Atomicity Consistency Isolation Durability
```

5 PART 5 — Functions, Triggers, JSONB, Arrays & Optimization

5.1 SQL Functions

```SQL
CREATE FUNCTION add_numbers(a INTEGER, b INTEGER)
RETURNS INTEGER
LANGUAGE SQL
AS $$
    SELECT a + b;
$$;

SELECT add_numbers(10, 20);  -- 30

CREATE FUNCTION get_annual_salary(monthly NUMERIC)
RETURNS NUMERIC LANGUAGE SQL AS $$
    SELECT monthly * 12;
$$;

SELECT name, salary, get_annual_salary(salary) AS annual
FROM employees;
```

5.2 PL/pgSQL Functions (Procedural)

Complex logic, IF/ELSE, loops, variables ke liye.

```SQL
CREATE FUNCTION calculate_bonus(salary NUMERIC)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
BEGIN
    IF salary >= 100000 THEN
        RETURN salary * 0.20;
    ELSIF salary >= 50000 THEN
        RETURN salary * 0.10;
    ELSE
        RETURN salary * 0.05;
    END IF;
END;
$$;

SELECT calculate_bonus(120000);  -- 24000
```

5.3 Stored Procedures

```SQL
CREATE PROCEDURE update_salary(emp_id INTEGER, new_salary NUMERIC)
LANGUAGE SQL
AS $$
    UPDATE employees SET salary = new_salary WHERE employee_id = emp_id;
$$;

CALL update_salary(101, 75000);
```

Function	Procedure
Called with SELECT	Called with CALL
Can return value / set	Primarily side effects
Usable inside queries	Standalone operations
Good for calculations	Good for business processes

5.4 Triggers

Database event (INSERT/UPDATE/DELETE) par automatic action. Audit logs, derived values, validation ke liye.

```sql
CREATE FUNCTION log_employee_update()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO employee_audit (employee_id, action_time, old_salary, new_salary)
    VALUES (NEW.employee_id, CURRENT_TIMESTAMP, OLD.salary, NEW.salary);
    RETURN NEW;
END;
$$;

CREATE TRIGGER employee_update_trigger
AFTER UPDATE OF salary ON employees
FOR EACH ROW
EXECUTE FUNCTION log_employee_update();
```

NEW = new row values, OLD = previous row values. BEFORE triggers values modify kar sakte hain; AFTER typically logging ke liye.

5.5 JSONB in Depth

```sql
CREATE TABLE candidates (
    candidate_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100),
    profile JSONB
);

INSERT INTO candidates (name, profile) VALUES
('Gaurav', '{"skills": ["Python","SQL","ML"], "experience": 2, "city": "Delhi"}');

-- Extract
SELECT profile->'skills'        FROM candidates;  -- JSON
SELECT profile->>'skills'       FROM candidates;  -- text
SELECT profile->'skills'->0     FROM candidates;  -- first skill as JSON
SELECT profile->>'city'         FROM candidates;

-- Filter
SELECT * FROM candidates WHERE profile->>'experience' = '2';
SELECT * FROM candidates WHERE (profile->>'experience')::int >= 2;
SELECT * FROM candidates WHERE profile @> '{"experience": 2}';
SELECT * FROM candidates WHERE profile @> '{"skills": ["Python"]}';

-- GIN index for containment queries
CREATE INDEX idx_profile ON candidates USING GIN (profile);
```

TIP

-> returns JSON, ->> returns text. @> containment operator JSONB ke liye bahut powerful hai. GIN index containment / existence queries ko tez banata hai.

5.6 Arrays in Depth

```sql
CREATE TABLE students (
    student_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100),
    skills TEXT[]
);

INSERT INTO students (name, skills) VALUES
('Gaurav', ARRAY['Python', 'SQL', 'Machine Learning']);

SELECT skills[1] FROM students;                -- 1-based
SELECT * FROM students WHERE 'Python' = ANY(skills);
SELECT * FROM students WHERE skills @> ARRAY['Python'];
SELECT name, UNNEST(skills) AS skill FROM students;
SELECT array_length(skills, 1) FROM students;
SELECT array_append(skills, 'Docker') FROM students;
```

5.7 EXPLAIN & Query Plans

```sql
EXPLAIN SELECT * FROM customers WHERE email = 'abc@gmail.com';
EXPLAIN ANALYZE SELECT * FROM customers WHERE email = 'abc@gmail.com';
-- ANALYZE actually runs the query and shows real timings + row counts
```

Scan / Node	Meaning
Seq Scan	Full table read — OK for small tables / low selectivity
Index Scan	Index use + table heap fetch
Index Only Scan	Mostly answered from index (covering)
Bitmap Index Scan	Build bitmap then heap fetch — good for many matches
Nested Loop / Hash / Merge Join	Join strategies

Planner cost, statistics, table size, selectivity dekh kar plan choose karta hai. Index hone ka matlab hamesha index use hoga — nahi.

5.8 Query Optimization Checklist

- SELECT * avoid karo — sirf chahiye columns
- WHERE / JOIN / ORDER BY columns par suitable indexes
- Large tables par early filter (date range, status)
- Functions on indexed columns (LOWER(email)) se index skip ho sakta hai — expression index banao
- EXPLAIN ANALYZE se actual plan verify karo
- Unnecessary DISTINCT / ORDER BY hatao
- JOIN order aur JOIN type (INNER vs LEFT) soch ke likho
- Statistics update: ANALYZE table_name;

5.9 Normalization Basics

Goal: duplication kam karo, update anomalies avoid karo, relationships clearly model karo.

- 1NF — Atomic values, no repeating groups (skills comma-list → separate rows or array carefully)
- 2NF — 1NF + no partial dependency on part of composite key
- 3NF — 2NF + non-key attributes depend only on key (not on other non-keys)

E-commerce: customers | products | orders | order_items — classic normalized design.

5.10 UPSERT & RETURNING

```sql
-- UPSERT (INSERT or UPDATE on conflict)
INSERT INTO users (email, name)
VALUES ('abc@gmail.com', 'Gaurav')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name;

-- RETURNING: get values back from write
INSERT INTO customers (name, email)
VALUES ('Gaurav', 'g@gmail.com')
RETURNING customer_id, created_at;

UPDATE products SET stock = stock - 1
WHERE product_id = 501 AND stock > 0
RETURNING product_id, stock;

DELETE FROM sessions WHERE expired_at < NOW()
RETURNING session_id;
```

5.11 Real-World E-Commerce Schema

```sql
CREATE TABLE customers (
    customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    product_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    category VARCHAR(100),
    price NUMERIC(10,2) CHECK (price >= 0),
    stock INTEGER CHECK (stock >= 0)
);

CREATE TABLE orders (
    order_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    order_date TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE order_items (
    order_item_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(order_id),
    product_id INTEGER NOT NULL REFERENCES products(product_id),
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0)
);
```

5.12 Real-World Interview Queries

```sql
-- Total spending per customer
SELECT c.customer_id, c.name,
       SUM(oi.quantity * oi.price) AS total_spending
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name
ORDER BY total_spending DESC;

-- Top 3 customers
... ORDER BY total_spending DESC LIMIT 3;

-- Products never ordered
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;

-- Category-wise average price
SELECT category, AVG(price) FROM products GROUP BY category;

-- Customers with > 5 orders
SELECT customer_id, COUNT(*) FROM orders
GROUP BY customer_id HAVING COUNT(*) > 5;

-- Top 2 products per category
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY category ORDER BY price DESC
    ) AS rn FROM products
)
SELECT * FROM ranked WHERE rn <= 2;
```

5.13 Complete Learning Roadmap

Level	Topics
1. Fundamentals	DB, tables, rows, columns, schemas
2. Data Types	INT, NUMERIC, VARCHAR, DATE, JSONB, UUID, ARRAY
3. Constraints	PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT
4. CRUD	INSERT, SELECT, UPDATE, DELETE, ALTER
5. Filtering	WHERE, IN, BETWEEN, LIKE, ILIKE, NULL
6. Aggregation	COUNT, SUM, AVG, MIN, MAX
7. Grouping	GROUP BY, HAVING
8. JOINs	INNER, LEFT, RIGHT, FULL, CROSS, SELF
9. Subqueries	Scalar, IN, EXISTS, correlated
10. CTE	WITH, recursive CTE
11. Windows	ROW_NUMBER, RANK, LAG, LEAD, frames
12. Views	VIEW, MATERIALIZED VIEW
13. Indexes	B-tree, composite, partial, GIN, expression
14. Transactions	BEGIN, COMMIT, ROLLBACK, SAVEPOINT, ACID
15–17. Code	Functions, Procedures, Triggers
18–19. Advanced types	JSONB, Arrays
20–22. Production	EXPLAIN, Optimization, Normalization, UPSERT

YOU ARE READY

Ab tumhare paas fundamentals se lekar advanced/interview-level PostgreSQL ka solid, detailed base hai. Har part ke practice sets solve karo, real projects mein apply karo, aur EXPLAIN ANALYZE se plans dekhna seekho. Happy querying!

— End of PostgreSQL Complete Notes (Detailed Premium Edition) —